

UNIT – II

Geometrical Transformations

2D Transformations, Homogeneous Coordinates and Matrix Representation of 2D Transformations, Composition of 2D Transformations, The Window-to-Viewport Transformation, Efficiency, Matrix Representation of 3D Transformations, Transformations as a Change in Coordinate System.

2D TRANSFORMATION

Transformation means changing some graphics into something else by applying rules. We can have various types of transformations such as translation, scaling up or down, rotation, shearing, etc. When a transformation takes place on a 2D plane, it is called 2D transformation.

Transformations play an important role in computer graphics to reposition the graphics on the screen and change their size or orientation.

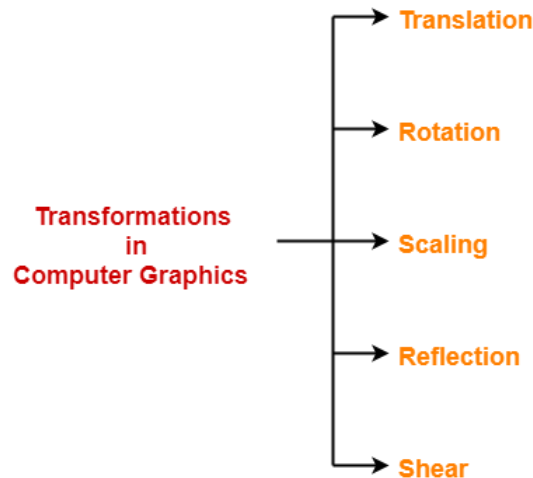
Homogenous Coordinates

To perform a sequence of transformation such as translation followed by rotation and scaling, we need to follow a sequential process –

- Translate the coordinates,
- Rotate the translated coordinates, and then
- Scale the rotated coordinates to complete the composite transformation.

To shorten this process, we have to use 3×3 transformation matrix instead of 2×2 transformation matrix. To convert a 2×2 matrix to 3×3 matrix, we have to add an extra dummy coordinate W.

In this way, we can represent the point by 3 numbers instead of 2 numbers, which is called Homogenous Coordinate system. In this system, we can represent all the transformation equations in matrix multiplication. Any Cartesian point $P(X, Y)$ can be converted to homogenous coordinates by $P'(X_h, Y_h, h)$.



2D Translation in Computer Graphics-

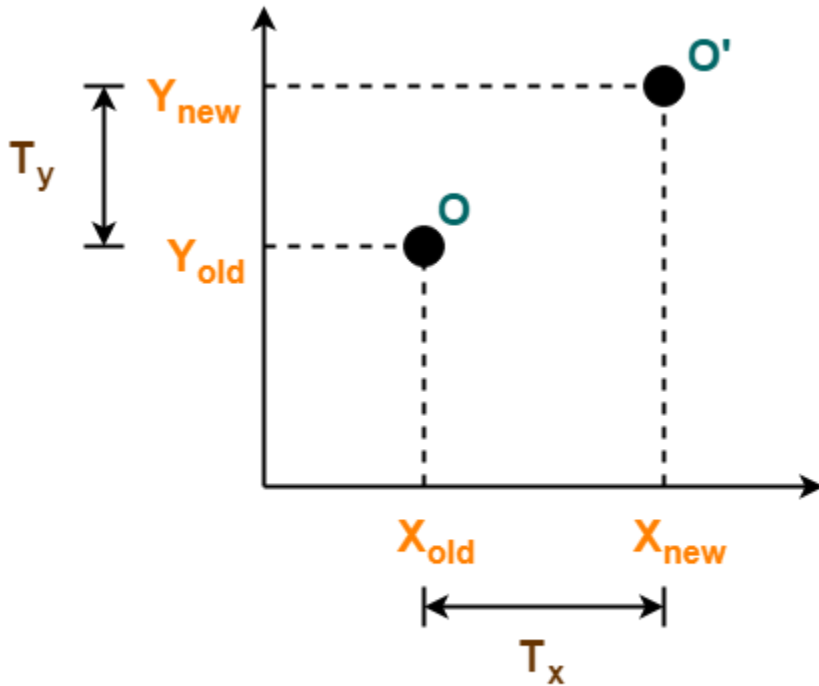
A translation moves an object to a different position on the screen. You can translate a point in 2D by adding translation coordinate (t_x, t_y) to the original coordinate X, Y to get the new coordinate X' ,

Let-

- Initial coordinates of the object $O = (X_{old}, Y_{old})$
- New coordinates of the object O after translation $= (X_{new}, Y_{new})$
- Translation vector or Shift vector $= (T_x, T_y)$

Given a Translation vector (T_x , T_y)-

- T_x defines the distance the X_{old} coordinate has to be moved.
- T_y defines the distance the Y_{old} coordinate has to be moved.



2D Translation in Computer Graphics

This translation is achieved by adding the translation coordinates to the old coordinates of the object as-

- $X_{new} = X_{old} + T_x$ (This denotes translation towards X axis)
- $Y_{new} = Y_{old} + T_y$ (This denotes translation towards Y axis)

In Matrix form, the above translation equations may be represented as-

$$\begin{bmatrix} X_{new} \\ Y_{new} \end{bmatrix} = \begin{bmatrix} X_{old} \\ Y_{old} \end{bmatrix} + \begin{bmatrix} T_x \\ T_y \end{bmatrix}$$

Translation Matrix

- The homogeneous coordinates representation of (X, Y) is $(X, Y, 1)$.

- Through this representation, all the transformations can be performed using matrix / vector multiplications. The above translation matrix may be represented as a 3 x 3 matrix as-

$$\begin{bmatrix} X_{\text{new}} \\ Y_{\text{new}} \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & T_x \\ 0 & 1 & T_y \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{\text{old}} \\ Y_{\text{old}} \\ 1 \end{bmatrix}$$

Translation Matrix
(Homogeneous Coordinates Representation)

EXAMPLE :

Given a square with coordinate points A(0, 3), B(3, 3), C(3, 0), D(0, 0). Apply the translation with distance 1 towards X axis and 1 towards Y axis. Obtain the new coordinates of the square.

Solution-

Given- Old coordinates of the square = A (0, 3), B(3, 3), C(3, 0), D(0, 0)

Translation vector = $(T_x, T_y) = (1, 1)$

For Coordinates A(0, 3)

Let the new coordinates of corner A = $(X_{\text{new}}, Y_{\text{new}})$.

Applying the translation equations, we have-

- $X_{\text{new}} = X_{\text{old}} + T_x = 0 + 1 = 1$
- $Y_{\text{new}} = Y_{\text{old}} + T_y = 3 + 1 = 4$

Thus, New coordinates of corner A = (1, 4).

For Coordinates B(3, 3)

Let the new coordinates of corner B = $(X_{\text{new}}, Y_{\text{new}})$.

Applying the translation equations, we have-

- $X_{\text{new}} = X_{\text{old}} + T_x = 3 + 1 = 4$
- $Y_{\text{new}} = Y_{\text{old}} + T_y = 3 + 1 = 4$

Thus, New coordinates of corner B = (4, 4).

For Coordinates C(3, 0)

Let the new coordinates of corner C = $(X_{\text{new}}, Y_{\text{new}})$.

Applying the translation equations, we have-

- $X_{\text{new}} = X_{\text{old}} + T_x = 3 + 1 = 4$
- $Y_{\text{new}} = Y_{\text{old}} + T_y = 0 + 1 = 1$

Thus, New coordinates of corner C = (4, 1).

For Coordinates D(0, 0)

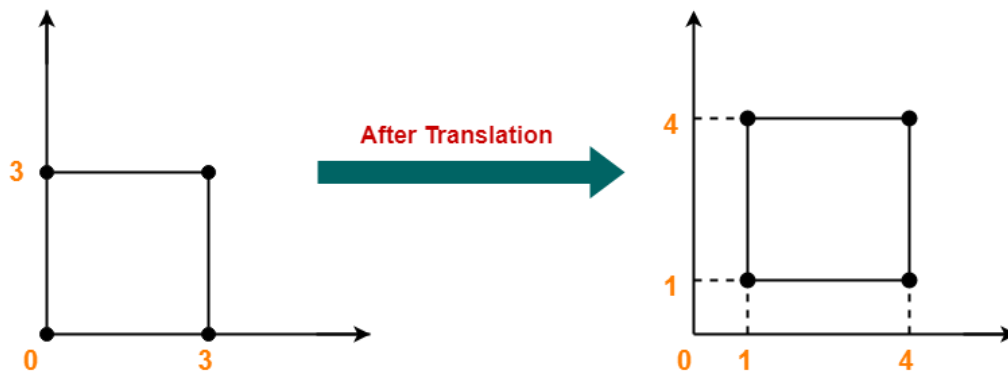
Let the new coordinates of corner D = $(X_{\text{new}}, Y_{\text{new}})$.

Applying the translation equations, we have-

- $X_{\text{new}} = X_{\text{old}} + T_x = 0 + 1 = 1$
- $Y_{\text{new}} = Y_{\text{old}} + T_y = 0 + 1 = 1$

Thus, New coordinates of corner D = (1, 1).

Thus, New coordinates of the square = A (1, 4), B(4, 4), C(4, 1), D(1, 1).



2D Scaling in Computer Graphics-

In computer graphics, scaling is a process of modifying or altering the size of objects.

- Scaling may be used to increase or reduce the size of object.
- Scaling subjects the coordinate points of the original object to change.
- Scaling factor determines whether the object size is to be increased or reduced.
- If scaling factor > 1 , then the object size is increased.
- If scaling factor < 1 , then the object size is reduced.

Consider a point object O has to be scaled in a 2D plane.

Let-

- Initial coordinates of the object O = $(X_{\text{old}}, Y_{\text{old}})$

- Scaling factor for X-axis = S_x
- Scaling factor for Y-axis = S_y
- New coordinates of the object O after scaling = (X_{new}, Y_{new})

This scaling is achieved by using the following scaling equations-

- $X_{new} = X_{old} \times S_x$
- $Y_{new} = Y_{old} \times S_y$

In Matrix form, the above scaling equations may be represented as-

$$\begin{bmatrix} X_{new} \\ Y_{new} \end{bmatrix} = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \times \begin{bmatrix} X_{old} \\ Y_{old} \end{bmatrix}$$

Scaling Matrix

For homogeneous coordinates, the above scaling matrix may be represented as a 3 x 3 matrix as-

$$\begin{bmatrix} X_{new} \\ Y_{new} \\ 1 \end{bmatrix} = \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{old} \\ Y_{old} \\ 1 \end{bmatrix}$$

Scaling Matrix
(Homogeneous Coordinates Representation)

EXAMPLE :

Given a square object with coordinate points A(0, 3), B(3, 3), C(3, 0), D(0, 0). Apply the scaling parameter 2 towards X axis and 3 towards Y axis and obtain the new coordinates of the object.

Solution-

- Old corner coordinates of the square = A (0, 3), B(3, 3), C(3, 0), D(0, 0)
- Scaling factor along X axis = 2
- Scaling factor along Y axis = 3

For Coordinates A(0, 3)

Let the new coordinates of corner A after scaling = $(X_{\text{new}}, Y_{\text{new}})$.

Applying the scaling equations, we have-

- $X_{\text{new}} = X_{\text{old}} \times S_x = 0 \times 2 = 0$
- $Y_{\text{new}} = Y_{\text{old}} \times S_y = 3 \times 3 = 9$

Thus, New coordinates of corner A after scaling = $(0, 9)$.

For Coordinates B(3, 3)

Let the new coordinates of corner B after scaling = $(X_{\text{new}}, Y_{\text{new}})$.

Applying the scaling equations, we have-

- $X_{\text{new}} = X_{\text{old}} \times S_x = 3 \times 2 = 6$
- $Y_{\text{new}} = Y_{\text{old}} \times S_y = 3 \times 3 = 9$

Thus, New coordinates of corner B after scaling = $(6, 9)$.

For Coordinates C(3, 0)

Let the new coordinates of corner C after scaling = $(X_{\text{new}}, Y_{\text{new}})$.

Applying the scaling equations, we have-

- $X_{\text{new}} = X_{\text{old}} \times S_x = 3 \times 2 = 6$
- $Y_{\text{new}} = Y_{\text{old}} \times S_y = 0 \times 3 = 0$

Thus, New coordinates of corner C after scaling = $(6, 0)$.

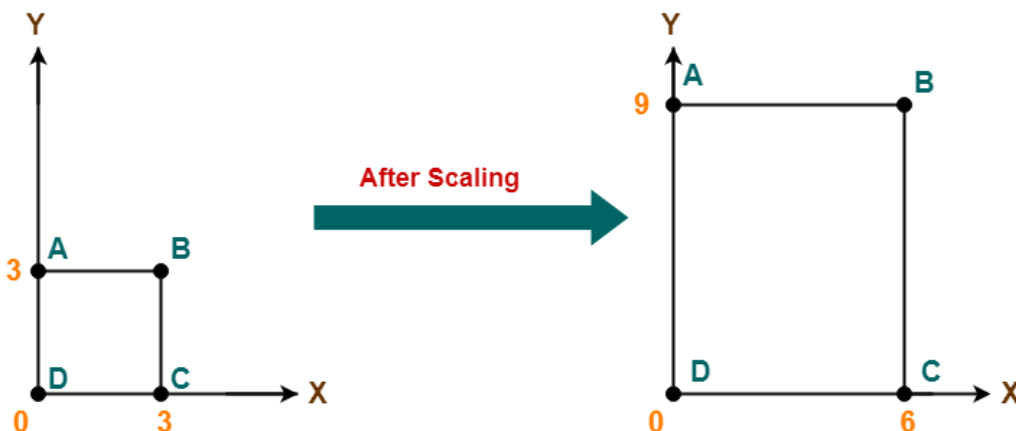
For Coordinates D(0, 0)

Let the new coordinates of corner D after scaling = $(X_{\text{new}}, Y_{\text{new}})$.

Applying the scaling equations, we have-

- $X_{\text{new}} = X_{\text{old}} \times S_x = 0 \times 2 = 0$
- $Y_{\text{new}} = Y_{\text{old}} \times S_y = 0 \times 3 = 0$

Thus, New coordinates of corner D after scaling = $(0, 0)$.



2D Rotation in Computer Graphics-

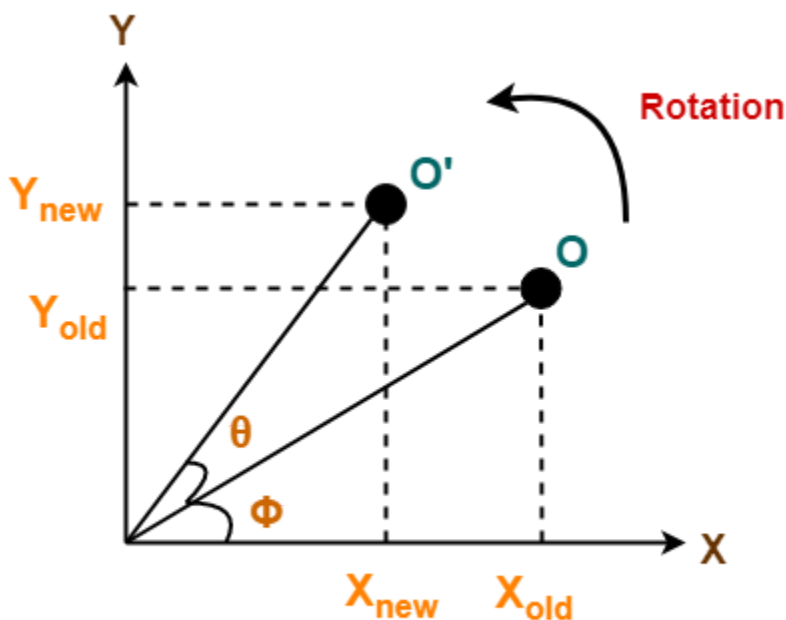
In Computer graphics,

2D Rotation is a process of rotating an object with respect to an angle in a two dimensional plane.

Consider a point object O has to be rotated from one angle to another in a 2D plane.

Let-

- Initial coordinates of the object O = (X_{old}, Y_{old})
- Initial angle of the object O with respect to origin = Φ
- Rotation angle = θ
- New coordinates of the object O after rotation = (X_{new}, Y_{new})



2D Rotation in Computer Graphics

This rotation is achieved by using the following rotation equations-

- $X_{new} = X_{old} \times \cos\theta - Y_{old} \times \sin\theta$
- $Y_{new} = X_{old} \times \sin\theta + Y_{old} \times \cos\theta$

In Matrix form, the above rotation equations may be represented as-

$$\begin{bmatrix} X_{\text{new}} \\ Y_{\text{new}} \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \times \begin{bmatrix} X_{\text{old}} \\ Y_{\text{old}} \end{bmatrix}$$

Rotation Matrix

For homogeneous coordinates, the above rotation matrix may be represented as a 3 x 3 matrix as-

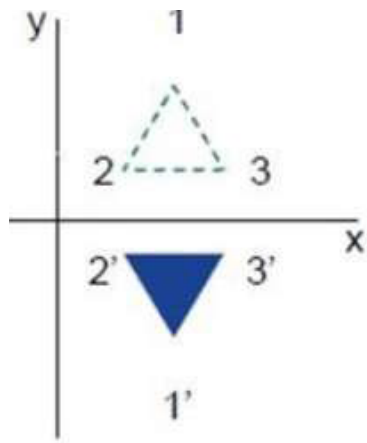
$$\begin{bmatrix} X_{\text{new}} \\ Y_{\text{new}} \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{\text{old}} \\ Y_{\text{old}} \\ 1 \end{bmatrix}$$

Rotation Matrix
(Homogeneous Coordinates Representation)

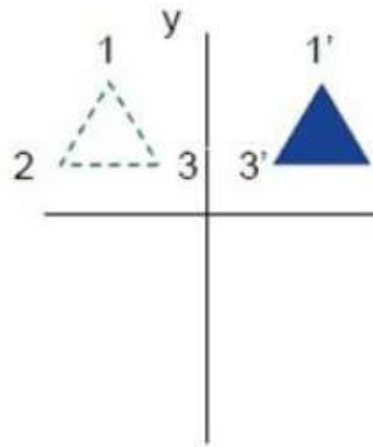
2D Reflection in Computer Graphics-

Reflection is the mirror image of original object. In other words, we can say that it is a rotation operation with 180°. In reflection transformation, the size of the object does not change. The following figures show reflections with respect to X and Y axes, and about the origin respectively.

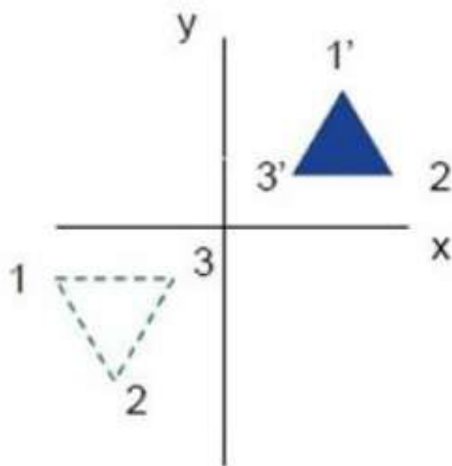
- Reflection is a kind of rotation where the angle of rotation is 180 degree.
- The reflected object is always formed on the other side of mirror.
- The size of reflected object is same as the size of original object.



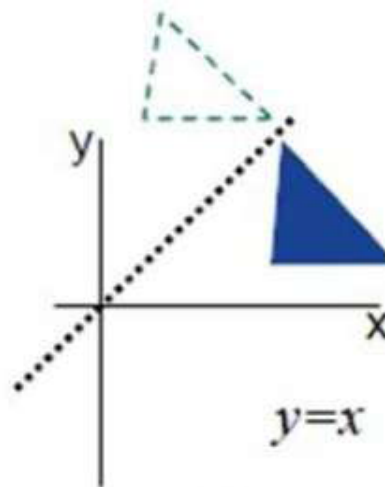
(a)



(b)



(c)



(d)

Consider a point object O has to be reflected in a 2D plane.

Let-

- Initial coordinates of the object O = $(X_{\text{old}}, Y_{\text{old}})$
- New coordinates of the reflected object O after reflection = $(X_{\text{new}}, Y_{\text{new}})$

Reflection On X-Axis:

This reflection is achieved by using the following reflection equations-

- $X_{\text{new}} = X_{\text{old}}$
- $Y_{\text{new}} = -Y_{\text{old}}$

In Matrix form, the above reflection equations may be represented as-

$$\begin{bmatrix} X_{\text{new}} \\ Y_{\text{new}} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \times \begin{bmatrix} X_{\text{old}} \\ Y_{\text{old}} \end{bmatrix}$$

Reflection Matrix
(Reflection Along X Axis)

For homogeneous coordinates, the above reflection matrix may be represented as a 3 x 3 matrix as-

$$\begin{bmatrix} X_{\text{new}} \\ Y_{\text{new}} \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{\text{old}} \\ Y_{\text{old}} \\ 1 \end{bmatrix}$$

Reflection Matrix
(Reflection Along X Axis)
(Homogeneous Coordinates Representation)

Reflection On Y-Axis:

This reflection is achieved by using the following reflection equations-

- $X_{\text{new}} = -X_{\text{old}}$
- $Y_{\text{new}} = Y_{\text{old}}$

In Matrix form, the above reflection equations may be represented as-

$$\begin{bmatrix} X_{\text{new}} \\ Y_{\text{new}} \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{\text{old}} \\ Y_{\text{old}} \end{bmatrix}$$

Reflection Matrix
(Reflection Along Y Axis)

For homogeneous coordinates, the above reflection matrix may be represented as a 3 x 3 matrix as-

$$\begin{bmatrix} X_{\text{new}} \\ Y_{\text{new}} \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{\text{old}} \\ Y_{\text{old}} \\ 1 \end{bmatrix}$$

Reflection Matrix
(Reflection Along Y Axis)
(Homogeneous Coordinates Representation)

EXAMPLE

Given a triangle with coordinate points A(3, 4), B(6, 4), C(5, 6). Apply the reflection on the X axis and obtain the new coordinates of the object.

Solution-

- Old corner coordinates of the triangle = A (3, 4), B(6, 4), C(5, 6)
- Reflection has to be taken on the X axis

For Coordinates A(3, 4)

Let the new coordinates of corner A after reflection = $(X_{\text{new}}, Y_{\text{new}})$.

Applying the reflection equations, we have-

- $X_{\text{new}} = X_{\text{old}} = 3$
- $Y_{\text{new}} = -Y_{\text{old}} = -4$

Thus, New coordinates of corner A after reflection = (3, -4).

For Coordinates B(6, 4)

Let the new coordinates of corner B after reflection = $(X_{\text{new}}, Y_{\text{new}})$.

Applying the reflection equations, we have-

- $X_{\text{new}} = X_{\text{old}} = 6$
- $Y_{\text{new}} = -Y_{\text{old}} = -4$

Thus, New coordinates of corner B after reflection = (6, -4).

For Coordinates C(5, 6)

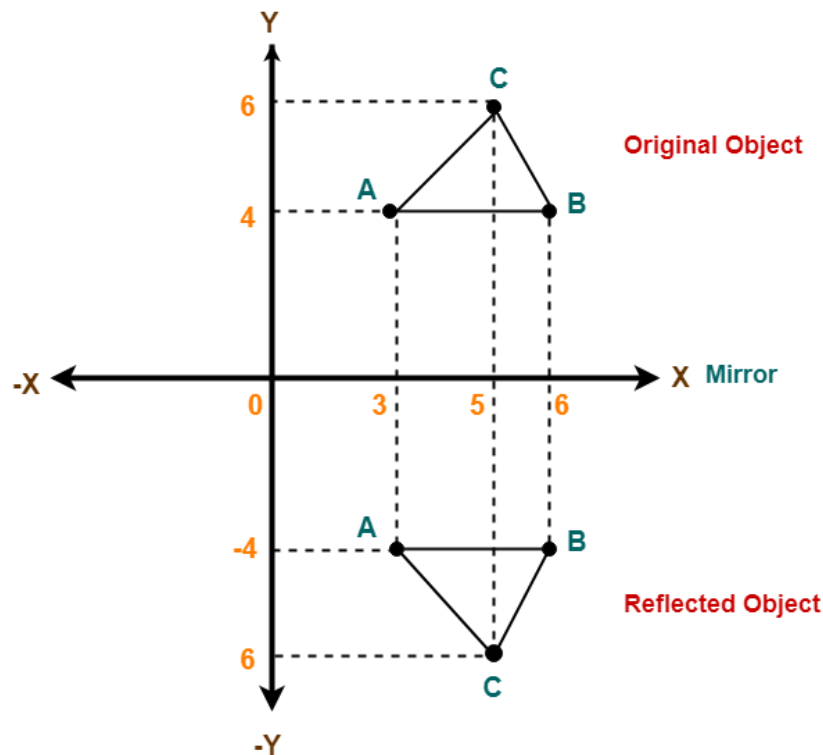
Let the new coordinates of corner C after reflection = $(X_{\text{new}}, Y_{\text{new}})$.

Applying the reflection equations, we have-

- $X_{\text{new}} = X_{\text{old}} = 5$
- $Y_{\text{new}} = -Y_{\text{old}} = -6$

Thus, New coordinates of corner C after reflection = (5, -6).

Thus, New coordinates of the triangle after reflection = A (3, -4), B(6, -4), C(5, -6).



2D Shearing in Computer Graphics-

In Computer graphics,

2D Shearing is an ideal technique to change the shape of an existing object in a two dimensional plane.

A transformation that slants the shape of an object is called the shear transformation. There are two shear transformations **X-Shear** and **Y-Shear**. One shifts X coordinates values and other shifts Y coordinate values. However; in both the cases only one coordinate changes its coordinates and other preserves its values. Shearing is also termed as **Skewing**.

In a two dimensional plane, the object size can be changed along X direction as well as Y direction.

So, there are two versions of shearing-

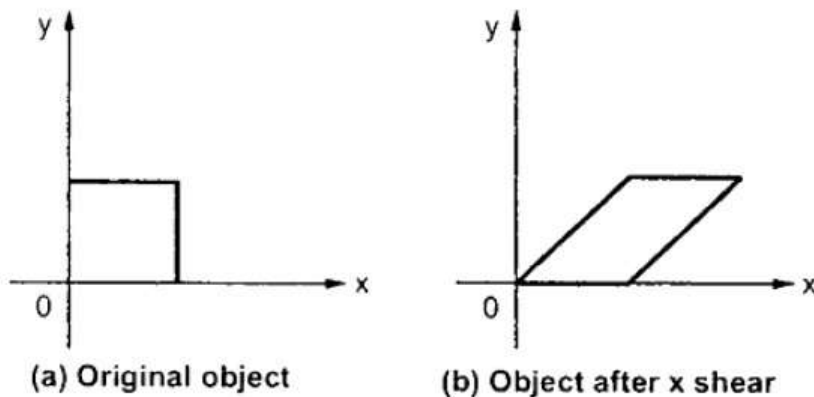
1. Shearing in X direction
2. Shearing in Y direction

Consider a point object O has to be sheared in a 2D plane. Now Let-

- Initial coordinates of the object $O = (X_{old}, Y_{old})$
- Shearing parameter towards X direction = Sh_x
- Shearing parameter towards Y direction = Sh_y
- New coordinates of the object O after shearing = (X_{new}, Y_{new})

Shearing in X Axis-

The X-Shear preserves the Y coordinate and changes are made to X coordinates, which causes the vertical lines to tilt right or left as shown in below figure.



Shearing in X axis is achieved by using the following shearing equations-

- $X_{new} = X_{old} + Sh_x \times Y_{old}$
- $Y_{new} = Y_{old}$

In Matrix form, the above shearing equations may be repressed

$$\begin{bmatrix} X_{\text{new}} \\ Y_{\text{new}} \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ Sh_x & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{\text{old}} \\ Y_{\text{old}} \\ 1 \end{bmatrix}$$

Shearing Matrix

(In X axis)

(Homogeneous Coordinates Representation)

nted as-

$$\begin{bmatrix} X_{\text{new}} \\ Y_{\text{new}} \end{bmatrix} = \begin{bmatrix} 1 & Sh_x \\ 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{\text{old}} \\ Y_{\text{old}} \end{bmatrix}$$

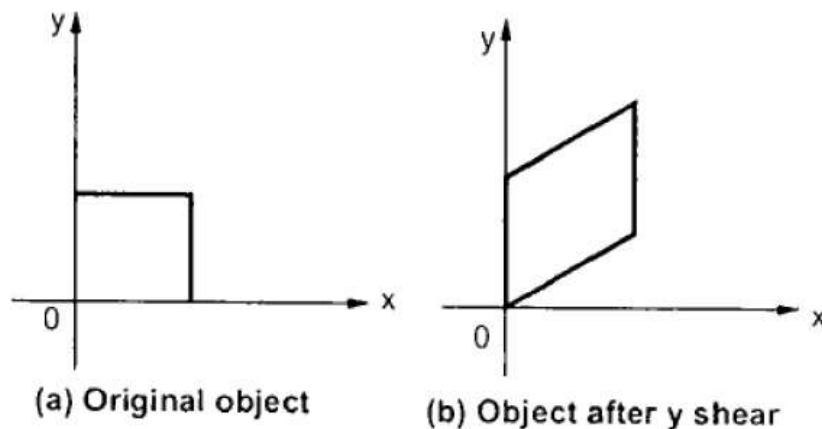
Shearing Matrix

(In X axis)

For homogeneous coordinates, the above shearing matrix may be represented as a 3 x 3 matrix as-

Shearing in Y Axis-

The Y-Shear preserves the X coordinates and changes the Y coordinates which causes the horizontal lines to transform into lines which slopes up or down as shown in the following figure.



Shearing in Y axis is achieved by using the following shearing equations-

- $X_{\text{new}} = X_{\text{old}}$
- $Y_{\text{new}} = Y_{\text{old}} + Sh_y \times X_{\text{old}}$

In Matrix form, the above shearing equations may be represented as-

$$\begin{bmatrix} X_{\text{new}} \\ Y_{\text{new}} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ Sh_y & 1 \end{bmatrix} \times \begin{bmatrix} X_{\text{old}} \\ Y_{\text{old}} \end{bmatrix}$$

Shearing Matrix
(In Y axis)

For homogeneous coordinates, the above shearing matrix may be represented as a 3 x 3 matrix as-

$$\begin{bmatrix} X_{\text{new}} \\ Y_{\text{new}} \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & Sh_y & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{\text{old}} \\ Y_{\text{old}} \\ 1 \end{bmatrix}$$

Shearing Matrix
(In Y axis)
(Homogeneous Coordinates Representation)

EXAMPLE

Given a triangle with points (1, 1), (0, 0) and (1, 0). Apply shear parameter 2 on X axis and 2 on Y axis and find out the new coordinates of the object.

Solution-

- Old corner coordinates of the triangle = A (1, 1), B(0, 0), C(1, 0)
- Shearing parameter towards X direction (Sh_x) = 2
- Shearing parameter towards Y direction (Sh_y) = 2

Shearing in X Axis-

For Coordinates A(1, 1)

Let the new coordinates of corner A after shearing = (X_{new} , Y_{new}).

Applying the shearing equations, we have-

- $X_{\text{new}} = X_{\text{old}} + Sh_x \times Y_{\text{old}} = 1 + 2 \times 1 = 3$

- $Y_{\text{new}} = Y_{\text{old}} = 1$

Thus, New coordinates of corner A after shearing = (3, 1).

For Coordinates B(0, 0)

Let the new coordinates of corner B after shearing = $(X_{\text{new}}, Y_{\text{new}})$.

Applying the shearing equations, we have-

- $X_{\text{new}} = X_{\text{old}} + Sh_x \times Y_{\text{old}} = 0 + 2 \times 0 = 0$
- $Y_{\text{new}} = Y_{\text{old}} = 0$

Thus, New coordinates of corner B after shearing = (0, 0).

For Coordinates C(1, 0)

Let the new coordinates of corner C after shearing = $(X_{\text{new}}, Y_{\text{new}})$.

Applying the shearing equations, we have-

- $X_{\text{new}} = X_{\text{old}} + Sh_x \times Y_{\text{old}} = 1 + 2 \times 0 = 1$
- $Y_{\text{new}} = Y_{\text{old}} = 0$

Thus, New coordinates of corner C after shearing = (1, 0).

Thus, New coordinates of the triangle after shearing in X axis = A (3, 1), B(0, 0), C(1, 0).

Shearing in Y Axis-

For Coordinates A(1, 1)

Let the new coordinates of corner A after shearing = $(X_{\text{new}}, Y_{\text{new}})$.

Applying the shearing equations, we have-

- $X_{\text{new}} = X_{\text{old}} = 1$
- $Y_{\text{new}} = Y_{\text{old}} + Sh_y \times X_{\text{old}} = 1 + 2 \times 1 = 3$

Thus, New coordinates of corner A after shearing = (1, 3).

For Coordinates B(0, 0)

Let the new coordinates of corner B after shearing = $(X_{\text{new}}, Y_{\text{new}})$.

Applying the shearing equations, we have-

- $X_{\text{new}} = X_{\text{old}} = 0$
- $Y_{\text{new}} = Y_{\text{old}} + Sh_y \times X_{\text{old}} = 0 + 2 \times 0 = 0$

Thus, New coordinates of corner B after shearing = (0, 0).

For Coordinates C(1, 0)

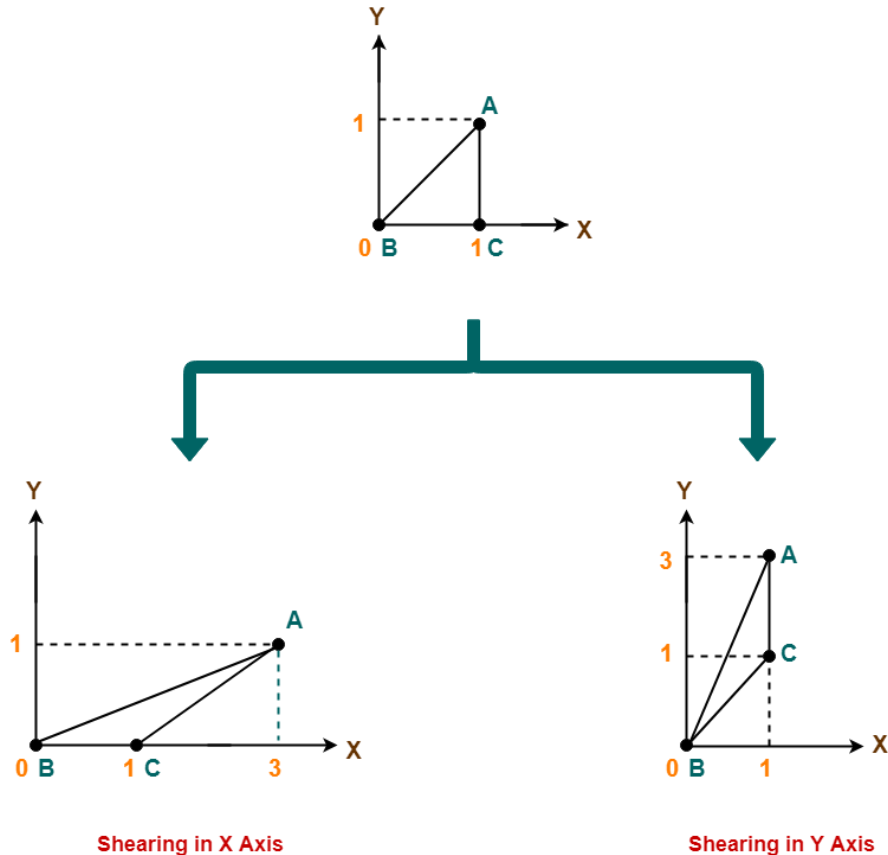
Let the new coordinates of corner C after shearing = $(X_{\text{new}}, Y_{\text{new}})$.

Applying the shearing equations, we have-

- $X_{\text{new}} = X_{\text{old}} = 1$
- $Y_{\text{new}} = Y_{\text{old}} + Sh_y \times X_{\text{old}} = 0 + 2 \times 1 = 2$

Thus, New coordinates of corner C after shearing = (1, 2).

Thus, New coordinates of the triangle after shearing in Y axis = A (1, 3), B(0, 0), C(1, 2).



Composite Transformation

If a transformation of the plane T1 is followed by a second plane transformation T2, then the result itself may be represented by a single transformation T which is the composition of T1 and T2 taken in that order. This is written as $T = T1 \cdot T2$.

Composite transformation can be achieved by concatenation of transformation matrices to obtain a combined transformation matrix.

A combined matrix –

$$[T][X] = [X] [T1] [T2] [T3] [T4] \dots [Tn]$$

Where [Ti] is any combination of

- Translation
- Scaling
- Shearing
- Rotation
- Reflection

The change in the order of transformation would lead to different results, as in general matrix multiplication is not cumulative, that is $[A] \cdot [B] \neq [B] \cdot [A]$ and the order of multiplication. The basic purpose of composing transformations is to gain efficiency by applying a single composed transformation to a point, rather than applying a series of transformation, one after another.

For example, to rotate an object about an arbitrary point (Xp, Yp), we have to carry out three steps

– Translate point (Xp, Yp) to the origin.

Rotate it about the origin.

Finally, translate the center of rotation back where it belonged.

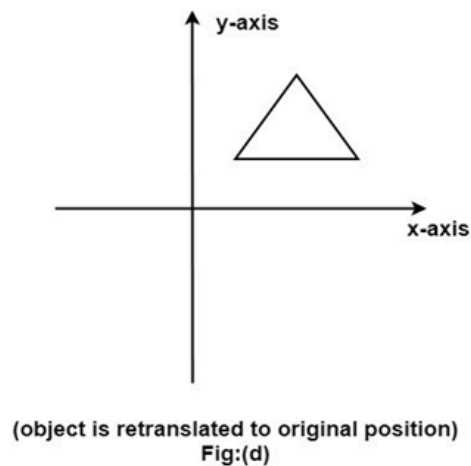
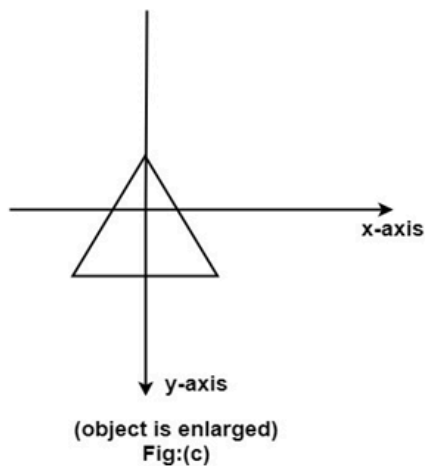
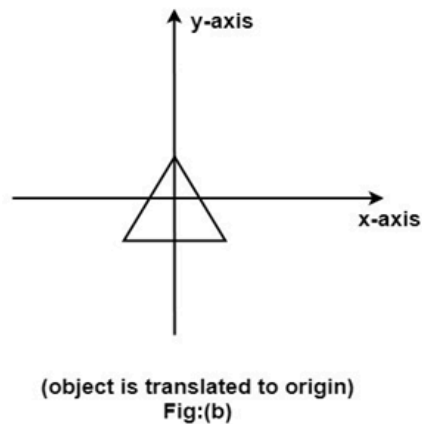
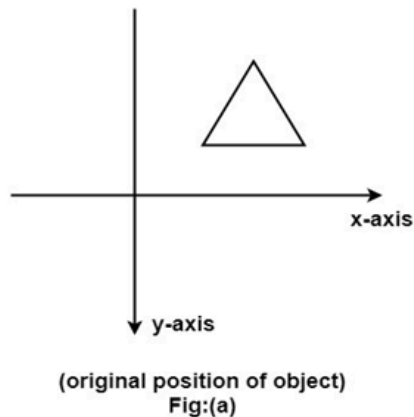
Step1: The object is kept at its position as in fig (a)

Step2: The object is translated so that its center coincides with the origin as in fig (b)

Step3: Scaling of an object by keeping the object at origin is done in fig (c)

Step4: Again translation is done. This second translation is called a reverse translation. It will position the object at the origin location.

Above transformation can be represented as $T_V \cdot S T_V^{-1}$



Advantage of composition or concatenation of matrix:

1. It transformations become compact.
2. The number of operations will be reduced.
3. Rules used for defining transformation in form of equations are complex as compared to matrix

Composition of two translations:

Let t_1, t_2, t_3, t_4 are translation vectors. They are two translations P_1 and P_2 . The matrix of P_1 and P_2 given below. The P_1 and P_2 are represented using Homogeneous matrices and P will be the final transformation matrix obtained after multiplication.

$$P_1 = \begin{bmatrix} 1 & 0 & t_1 \\ 0 & 1 & t_2 \\ 0 & 0 & 1 \end{bmatrix} \quad P_2 = \begin{bmatrix} 1 & 0 & t_3 \\ 0 & 1 & t_4 \\ 0 & 0 & 1 \end{bmatrix}$$

$$P = \begin{bmatrix} 1 & 0 & t_1 + t_2 \\ 0 & 1 & t_3 + t_4 \\ 0 & 0 & 1 \end{bmatrix}$$

Above resultant matrix show that two successive translations are additive.

Composition of two Rotations: Two Rotations are also additive

$$R(\alpha) R(\beta) = R(\alpha + \beta)$$

Composition of two Scaling: The composition of two scaling is multiplicative. Let S_{11} and S_{12} are matrix to be multiplied.

$$S_{11} = \begin{bmatrix} S_1 & 0 & 0 \\ 0 & S_2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$S_{12} = \begin{bmatrix} S_3 & 0 & 0 \\ 0 & S_4 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

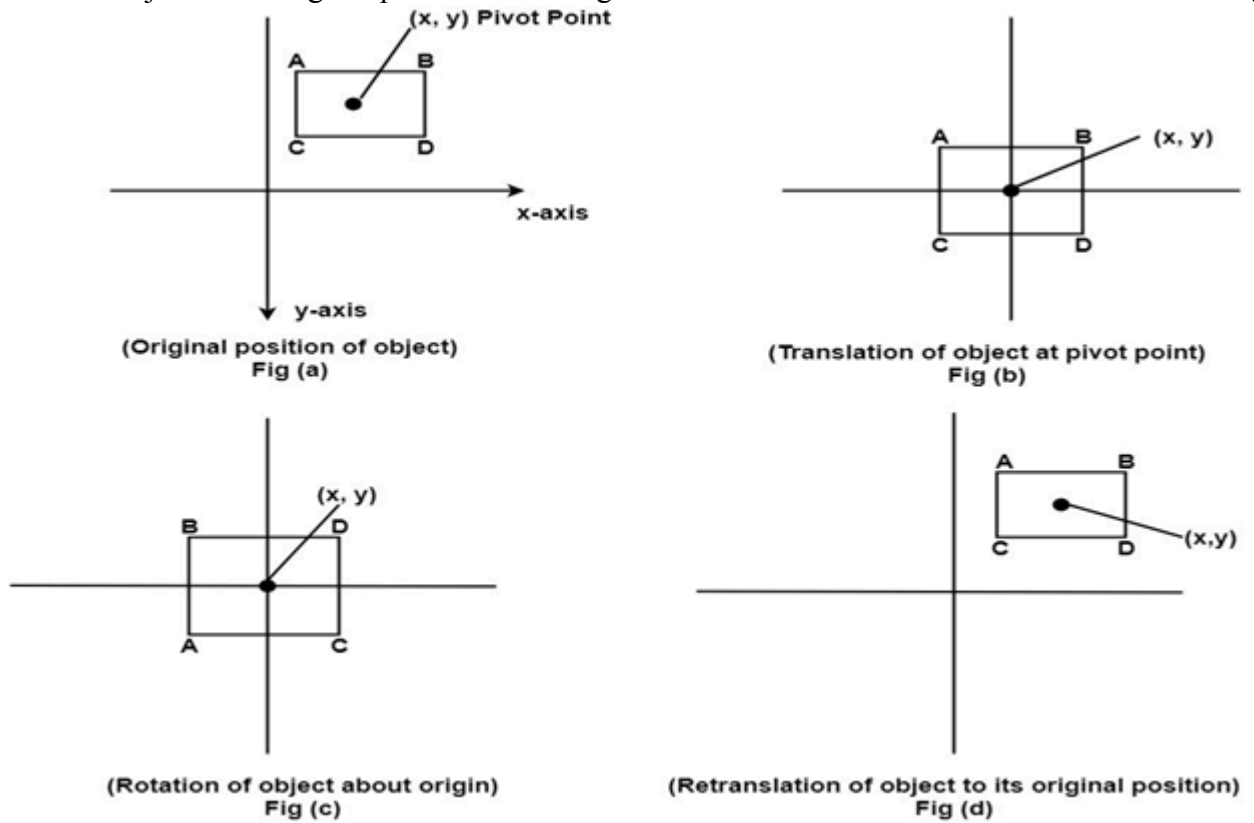
$$S = S_{11} * S_{12} = \begin{bmatrix} S_1 * S_3 & 0 & 0 \\ 0 & S_2 * S_4 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

General Pivot Point Rotation or Rotation about Fixed Point

For it first of all rotate function is used. Sequences of steps are given below for rotating an object about origin.

1. Translate object to origin from its original position as shown in fig (b)
2. Rotate the object about the origin as shown in fig (c).

3. Translate the object to its original position from origin. It is called as reverse translation as shown in fig



(d).

The matrix multiplication of above 3 steps is given below

$$\begin{bmatrix} 1 & 0 & x \\ 0 & 1 & y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -x \\ 0 & 1 & -y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & -\sin\theta & x(1 - \cos\theta) + y\sin\theta \\ \sin\theta & \cos\theta & y(1 - \cos\theta) - x\sin\theta \\ 0 & 0 & 1 \end{bmatrix}$$

Scaling relative to fixed point:

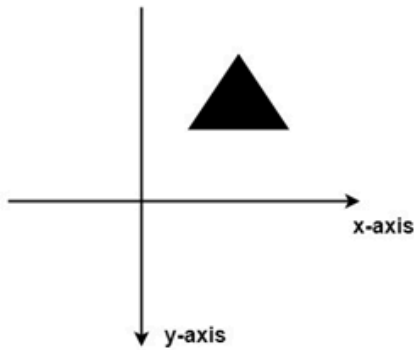
For this following steps are performed:

Step1: The object is kept at desired location as shown in fig (a)

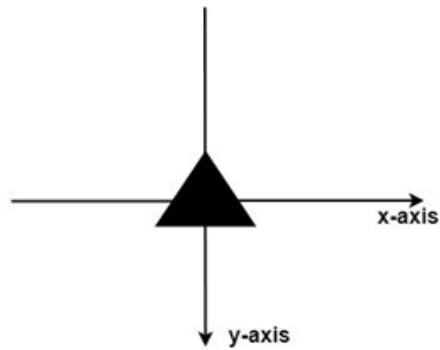
Step2: The object is translated so that its center coincides with origin as shown in fig (b)

Step3: Scaling of object by keeping object at origin is done as shown in fig (c)

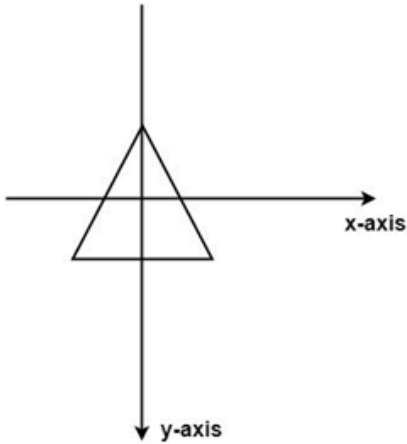
Step4: Again translation is done. This translation is called as reverse translation.



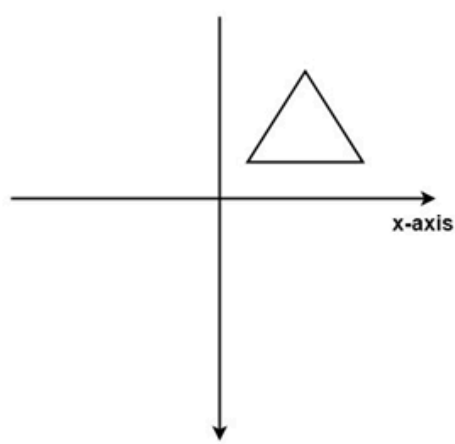
(Original position of object)
Fig (a)



(Object is translated to origin)
Fig (b)



(Object is enlarged by process of scaling)
Fig (c)

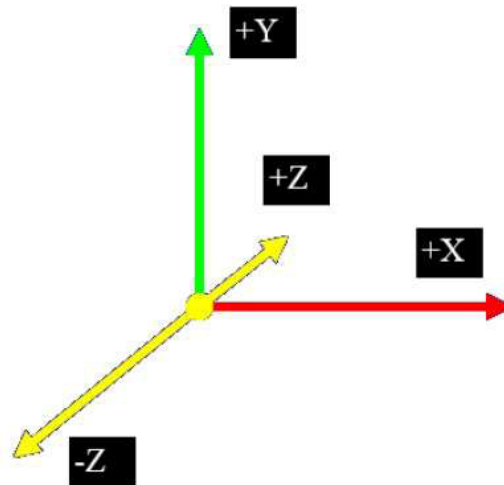


(Object is retranslated to original position)
Fig (d)

3D COMPUTER GRAPHICS

In the 2D system, we use only two coordinates X and Y but in 3D, an extra coordinate Z is added. 3D graphics techniques and their application are fundamental to the entertainment, games, and computer-aided design industries. It is a continuing area of research in scientific visualization. Furthermore, 3D graphics components are now a part of almost every personal computer and, although traditionally intended for graphics-intensive software such as games, they are increasingly being used by other applications.

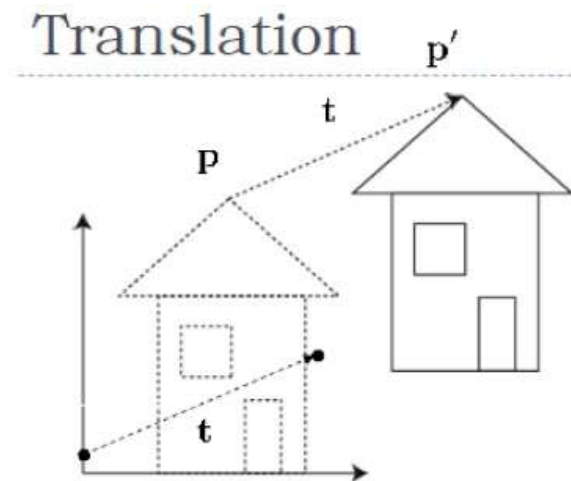
- 3D Transformations take place in a three dimensional plane.
- 3D Transformations are important and a bit more complex than 2D Transformations.
- Transformations are helpful in changing the position, size, orientation, shape etc of the object.



Translation

In 3D translation, we transfer the Z coordinate along with the X and Y coordinates. The process for translation in 3D is similar to 2D translation. A translation moves an object into a different position on the screen.

The following figure shows the effect of translation –

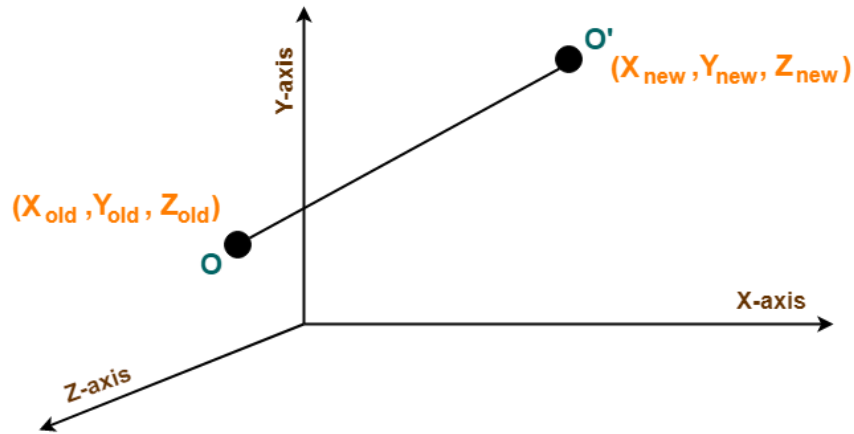


Let-

- Initial coordinates of the object $O = (X_{old}, Y_{old}, Z_{old})$
- New coordinates of the object O after translation $= (X_{new}, Y_{new}, Z_{old})$
- Translation vector or Shift vector $= (T_x, T_y, T_z)$

Given a Translation vector (T_x, T_y, T_z) -

- T_x defines the distance the X_{old} coordinate has to be moved.
- T_y defines the distance the Y_{old} coordinate has to be moved.
- T_z defines the distance the Z_{old} coordinate has to be moved.



3D Translation in Computer Graphics

This translation is achieved by adding the translation coordinates to the old coordinates of the object as-

- $X_{\text{new}} = X_{\text{old}} + T_x$ (This denotes translation towards X axis)
- $Y_{\text{new}} = Y_{\text{old}} + T_y$ (This denotes translation towards Y axis)
- $Z_{\text{new}} = Z_{\text{old}} + T_z$ (This denotes translation towards Z axis)

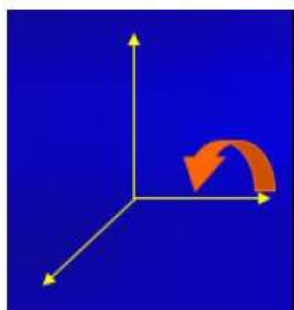
In Matrix form, the above translation equations may be represented as-

$$\begin{bmatrix} X_{\text{new}} \\ Y_{\text{new}} \\ Z_{\text{new}} \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & T_x \\ 0 & 1 & 0 & T_y \\ 0 & 0 & 1 & T_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{\text{old}} \\ Y_{\text{old}} \\ Z_{\text{old}} \\ 1 \end{bmatrix}$$

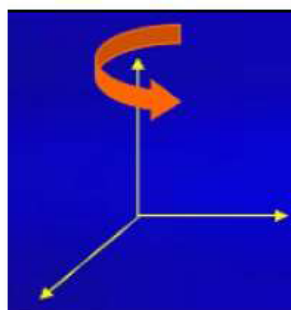
3D Translation Matrix

Rotation

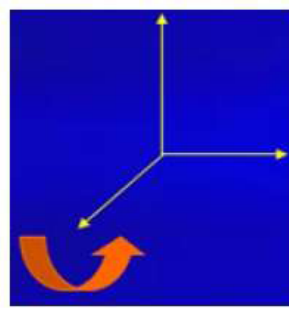
3D rotation is not same as 2D rotation. In 3D rotation, we have to specify the angle of rotation along with the axis of rotation. We can perform 3D rotation about X, Y, and Z axes. They are represented in the matrix form as below –



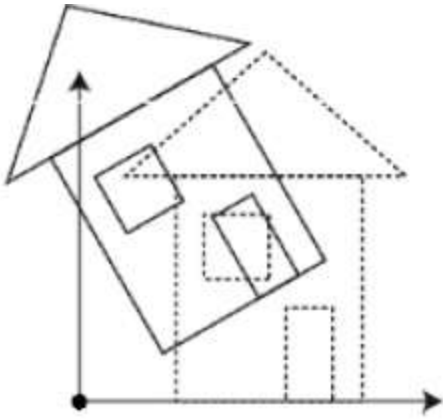
Rotation about x-axis



Rotation about y-axis



Rotation about z-axis



Consider a point object O has to be rotated from one angle to another in a 3D plane. Let-

- Initial coordinates of the object O = $(X_{old}, Y_{old}, Z_{old})$
- Initial angle of the object O with respect to origin = Φ
- Rotation angle = θ
- New coordinates of the object O after rotation = $(X_{new}, Y_{new}, Z_{new})$

In 3 dimensions, there are 3 possible types of rotation-

- X-axis Rotation
- Y-axis Rotation
- Z-axis Rotation

For X-Axis Rotation-

This rotation is achieved by using the following rotation equations-

- $X_{new} = X_{old}$
- $Y_{new} = Y_{old} \times \cos\theta - Z_{old} \times \sin\theta$
- $Z_{new} = Y_{old} \times \sin\theta + Z_{old} \times \cos\theta$

In Matrix form, the above rotation equations may be represented as-

$$\begin{bmatrix} X_{new} \\ Y_{new} \\ Z_{new} \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{old} \\ Y_{old} \\ Z_{old} \\ 1 \end{bmatrix}$$

3D Rotation Matrix
(For X-Axis Rotation)

For Y-Axis Rotation-

This rotation is achieved by using the following rotation equations-

- $X_{\text{new}} = Z_{\text{old}} \times \sin\theta + X_{\text{old}} \times \cos\theta$
- $Y_{\text{new}} = Y_{\text{old}}$
- $Z_{\text{new}} = Y_{\text{old}} \times \cos\theta - X_{\text{old}} \times \sin\theta$

In Matrix form, the above rotation equations may be represented as-

$$\begin{bmatrix} X_{\text{new}} \\ Y_{\text{new}} \\ Z_{\text{new}} \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{\text{old}} \\ Y_{\text{old}} \\ Z_{\text{old}} \\ 1 \end{bmatrix}$$

3D Rotation Matrix
(For Y-Axis Rotation)

For Z-Axis Rotation-

This rotation is achieved by using the following rotation equations-

- $X_{\text{new}} = X_{\text{old}} \times \cos\theta - Y_{\text{old}} \times \sin\theta$
- $Y_{\text{new}} = X_{\text{old}} \times \sin\theta + Y_{\text{old}} \times \cos\theta$
- $Z_{\text{new}} = Z_{\text{old}}$

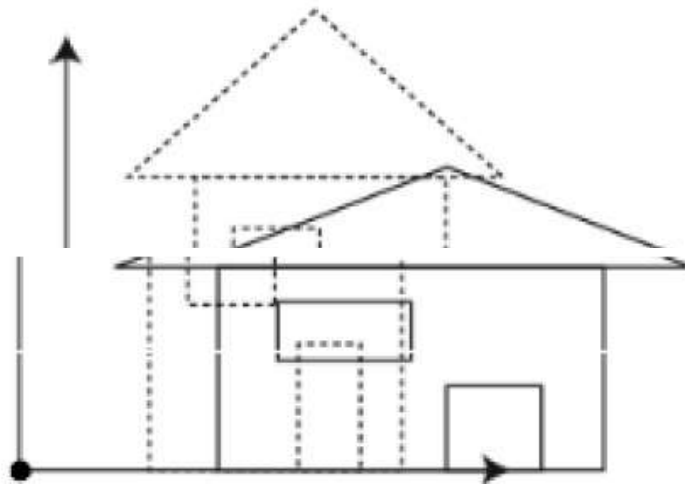
In Matrix form, the above rotation equations may be represented as-

$$\begin{bmatrix} X_{\text{new}} \\ Y_{\text{new}} \\ Z_{\text{new}} \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{\text{old}} \\ Y_{\text{old}} \\ Z_{\text{old}} \\ 1 \end{bmatrix}$$

3D Rotation Matrix
(For Z-Axis Rotation)

Scaling

You can change the size of an object using scaling transformation. In the scaling process, you either expand or compress the dimensions of the object. Scaling can be achieved by multiplying the original coordinates of the object with the scaling factor to get the desired result. The following figure shows the effect of 3D scaling –



- Scaling may be used to increase or reduce the size of object.
- Scaling subjects the coordinate points of the original object to change.
- Scaling factor determines whether the object size is to be increased or reduced.
- If scaling factor > 1 , then the object size is increased.
- If scaling factor < 1 , then the object size is reduced.

Consider a point object O has to be scaled in a 3D plane.

Let-

- Initial coordinates of the object $O = (X_{old}, Y_{old}, Z_{old})$
- Scaling factor for X-axis = S_x
- Scaling factor for Y-axis = S_y
- Scaling factor for Z-axis = S_z
- New coordinates of the object O after scaling = $(X_{new}, Y_{new}, Z_{new})$

This scaling is achieved by using the following scaling equations-

- $X_{new} = X_{old} \times S_x$
- $Y_{new} = Y_{old} \times S_y$
- $Z_{new} = Z_{old} \times S_z$

In Matrix form, the above scaling equations may be represented as-

$$\begin{bmatrix} X_{new} \\ Y_{new} \\ Z_{new} \\ 1 \end{bmatrix} = \begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{old} \\ Y_{old} \\ Z_{old} \\ 1 \end{bmatrix}$$

3D Scaling Matrix

Reflection

- Reflection is a kind of rotation where the angle of rotation is 180 degree.
- The reflected object is always formed on the other side of mirror.
- The size of reflected object is same as the size of original object.

Consider a point object O has to be reflected in a 3D plane.

Let-

- Initial coordinates of the object O = $(X_{old}, Y_{old}, Z_{old})$
- New coordinates of the reflected object O after reflection = $(X_{new}, Y_{new}, Z_{new})$

In 3 dimensions, there are 3 possible types of reflection-

- Reflection relative to XY plane
- Reflection relative to YZ plane
- Reflection relative to XZ plane

Reflection Relative to XY Plane:

This reflection is achieved by using the following reflection equations-

- $X_{new} = X_{old}$
- $Y_{new} = Y_{old}$
- $Z_{new} = -Z_{old}$

In Matrix form, the above reflection equations may be represented as-

$$\begin{bmatrix} X_{new} \\ Y_{new} \\ Z_{new} \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{old} \\ Y_{old} \\ Z_{old} \\ 1 \end{bmatrix}$$

3D Reflection Matrix
(Reflection Relative to XY plane)

Reflection Relative to YZ Plane:

This reflection is achieved by using the following reflection equations-

- $X_{new} = -X_{old}$

- $Y_{\text{new}} = Y_{\text{old}}$
- $Z_{\text{new}} = Z_{\text{old}}$

In Matrix form, the above reflection equations may be represented as-

$$\begin{bmatrix} X_{\text{new}} \\ Y_{\text{new}} \\ Z_{\text{new}} \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{\text{old}} \\ Y_{\text{old}} \\ Z_{\text{old}} \\ 1 \end{bmatrix}$$

3D Reflection Matrix
(Reflection Relative to YZ plane)

Reflection Relative to XZ Plane:

This reflection is achieved by using the following reflection equations-

- $X_{\text{new}} = X_{\text{old}}$
- $Y_{\text{new}} = -Y_{\text{old}}$
- $Z_{\text{new}} = Z_{\text{old}}$

In Matrix form, the above reflection equations may be represented as-

$$\begin{bmatrix} X_{\text{new}} \\ Y_{\text{new}} \\ Z_{\text{new}} \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{\text{old}} \\ Y_{\text{old}} \\ Z_{\text{old}} \\ 1 \end{bmatrix}$$

3D Reflection Matrix
(Reflection Relative to XZ plane)

Shearing

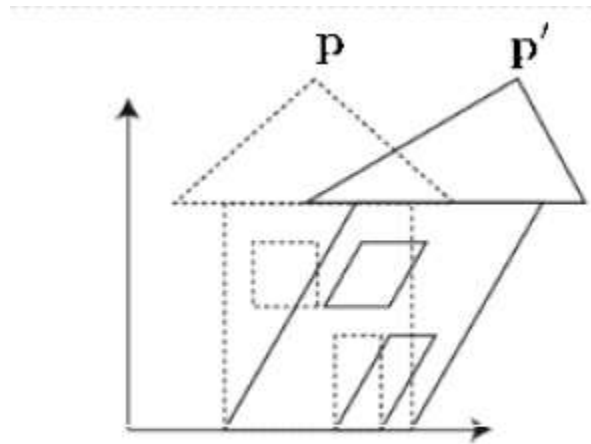
In a three dimensional plane, the object size can be changed along X direction, Y direction as well as Z direction.

So, there are three versions of shearing-

1. Shearing in X direction
- 2.
3. Shearing in Y direction
4. Shearing in Z direction

Consider a point object O has to be sheared in a 3D plane. Let-

- Initial coordinates of the object $O = (X_{old}, Y_{old}, Z_{old})$
- Shearing parameter towards X direction = Sh_x
- Shearing parameter towards Y direction = Sh_y
- Shearing parameter towards Z direction = Sh_z
- New coordinates of the object O after shearing = $(X_{new}, Y_{new}, Z_{new})$



Shearing in X Axis-

Shearing in X axis is achieved by using the following shearing equations-

- $X_{new} = X_{old}$
- $Y_{new} = Y_{old} + Sh_y \times X_{old}$
- $Z_{new} = Z_{old} + Sh_z \times X_{old}$

In Matrix form, the above shearing equations may be represented as-

$$\begin{bmatrix} X_{\text{new}} \\ Y_{\text{new}} \\ Z_{\text{new}} \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ Sh_y & 1 & 0 & 0 \\ Sh_z & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{\text{old}} \\ Y_{\text{old}} \\ Z_{\text{old}} \\ 1 \end{bmatrix}$$

3D Shearing Matrix
(In X axis)

Shearing in Y Axis-

Shearing in Y axis is achieved by using the following shearing equations-

- $X_{\text{new}} = X_{\text{old}} + Sh_x \times Y_{\text{old}}$
- $Y_{\text{new}} = Y_{\text{old}}$
- $Z_{\text{new}} = Z_{\text{old}} + Sh_z \times Y_{\text{old}}$

In Matrix form, the above shearing equations may be represented as-

$$\begin{bmatrix} X_{\text{new}} \\ Y_{\text{new}} \\ Z_{\text{new}} \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & Sh_x & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & Sh_z & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{\text{old}} \\ Y_{\text{old}} \\ Z_{\text{old}} \\ 1 \end{bmatrix}$$

3D Shearing Matrix
(In Y axis)

Shearing in Z Axis-

Shearing in Z axis is achieved by using the following shearing equations-

- $X_{\text{new}} = X_{\text{old}} + Sh_x \times Z_{\text{old}}$
- $Y_{\text{new}} = Y_{\text{old}} + Sh_y \times Z_{\text{old}}$
- $Z_{\text{new}} = Z_{\text{old}}$

In Matrix form, the above shearing equations may be represented as-

$$\begin{bmatrix} X_{\text{new}} \\ Y_{\text{new}} \\ Z_{\text{new}} \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & Sh_x & 0 \\ 0 & 1 & Sh_y & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{\text{old}} \\ Y_{\text{old}} \\ Z_{\text{old}} \\ 1 \end{bmatrix}$$

3D Shearing Matrix
(In Z axis)

TRIGO IDENTITIES (REFERENCE : <https://byjus.com/maths/trigonometric-identities/>)

- | | |
|--|--|
| • $\sin(-\theta) = -\sin \theta$ | $\cos(-\theta) = \cos \theta$ |
| • $\sin(90 - \theta) = \cos \theta$ | $\cos(90 - \theta) = \sin \theta$ |
| • $\sin(90 + \theta) = \cos \theta$ | $\cos(90 + \theta) = -\sin \theta$ |
| • $\sin(180 - \theta) = \sin \theta$ | $\cos(180 - \theta) = -\cos \theta$ |
| • $\sin(\alpha + \beta) = \sin \alpha \cos \beta + \cos \alpha \sin \beta$ | $\sin(\alpha - \beta) = \sin \alpha \cos \beta - \cos \alpha \sin \beta$ |
| • $\cos(\alpha + \beta) = \cos \alpha \cos \beta - \sin \alpha \sin \beta$ | $\cos(\alpha - \beta) = \cos \alpha \cos \beta + \sin \alpha \sin \beta$ |

WINDOW TO VIEWPORT

The method of selecting and enlarging a portion of a drawing is called **windowing**. The area chosen for this display is called a window. The window is selected by world-coordinate.

- Sometimes we are interested in some portion of the object and not in full object. So we will decide on an imaginary box. This box will enclose desired or interested area of the object. Such an imaginary box is called a window.
- **Viewport:** An area on display device to which a window is mapped [where it is to displayed]. Basically, the window is an area in object space. It encloses the object. After the user selects this, space is mapped on the whole area of the viewport. Almost all 2D and 3D graphics packages provide means of defining viewport size on the screen. It is possible to determine many viewports on different areas of display and view the same object in a different angle in each viewport.
- The size of the window is (0, 0) coordinate which is a bottom-left corner and toward right side until window encloses the desired area. Once the window is defined data outside the window is clipped before representing to screen coordinates. This process reduces the amount of data displaying signals.
- The window size of the Tektronix 4.14 tube in Imperial College contains 4.96 points horizontally and 3072 points vertically.
- **Viewing transformation or window to viewport transformation or windowing transformation:** The mapping of a part of a world-coordinate scene to device coordinates is referred to as a viewing transformation etc.

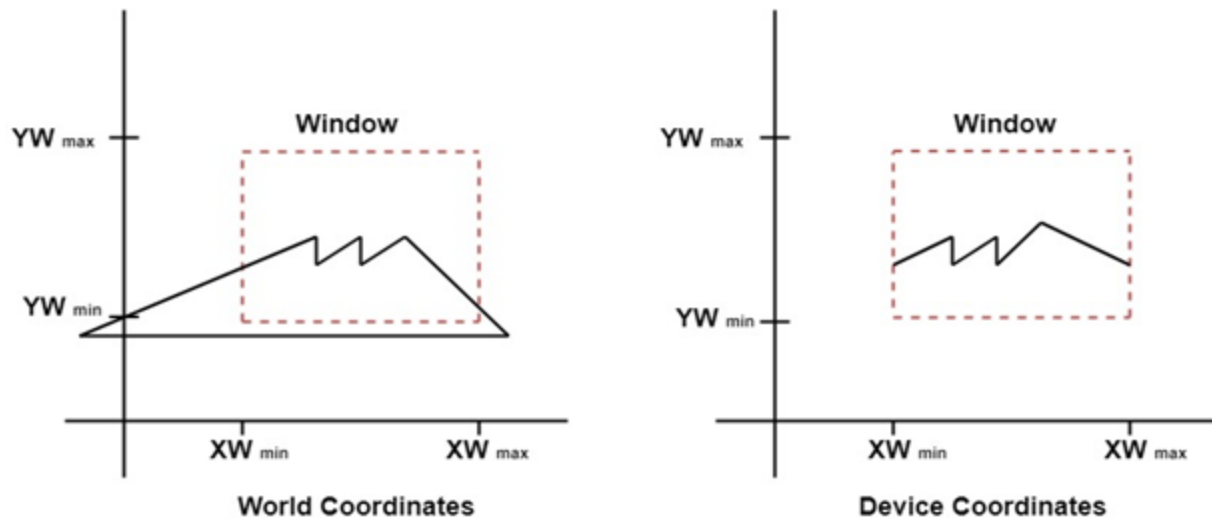


Fig: A viewing transformation using standard rectangles for the window and viewport.

-
- **Viewing transformation in several steps:**
- First, we construct the scene in world coordinate using the output primitives and attributes.
- To obtain a particular orientation, we can set up a 2-D viewing coordinate system in the window coordinate plane and define a window in viewing coordinates system.
- Once the viewing frame is established, are then transform description in world coordinates to viewing coordinates.
- Then, we define viewport in normalized coordinates (range from 0 to 1) and map the viewing coordinates description of the scene to normalized coordinates.
- At the final step, all parts of the picture that (i.e., outside the viewport are clipped, and the contents are transferred to device coordinates).

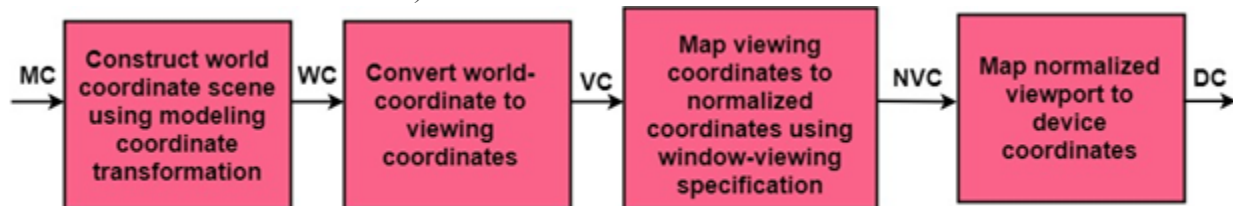


Fig: The two-dimensional viewing-transformation.

-
- **By changing the position of the viewport:** We can view objects at different locations on the display area of an output device as shown in fig:

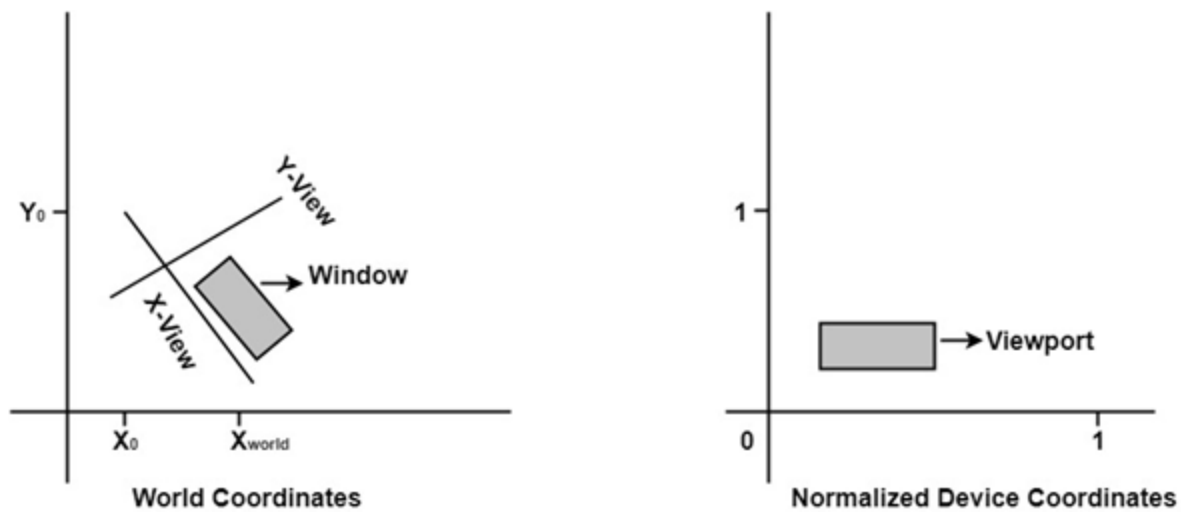


Fig: Setting up a rotated world window and corresponding normalized coordinate viewport.

-
- **By varying the size of viewports:** We can change the size and proportions of displayed objects. We can achieve zooming effects by successively mapping different-sized windows on a fixed-size viewport.
- As the windows are made smaller, we zoom in on some part of a scene to view details that are not shown with larger windows.

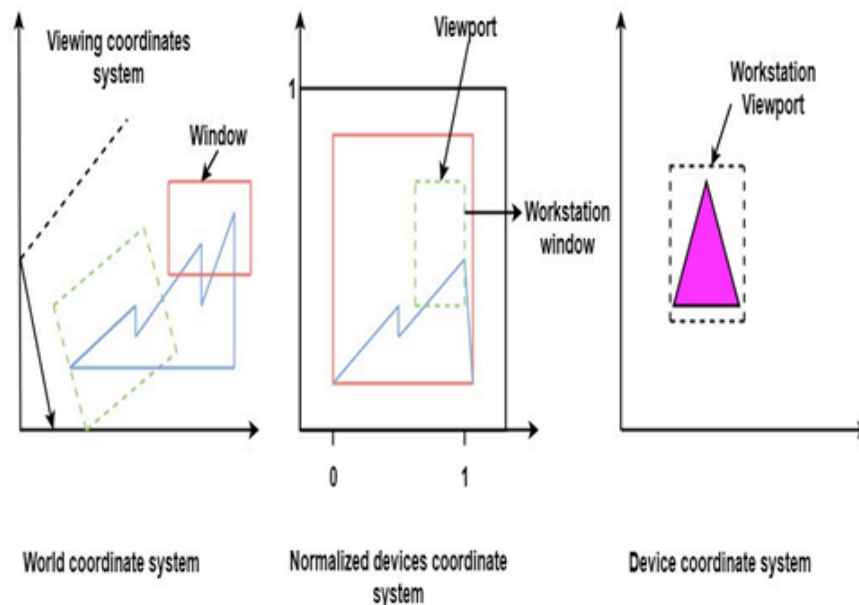


Fig: Zooming effects by mapping different-sized windows on a fixed-size viewport.

Computer Graphics Window to Viewport Co-ordinate Transformation

Once object description has been transmitted to the viewing reference frame, we choose the window extends in viewing coordinates and selects the viewport limits in normalized coordinates.

Object descriptions are then transferred to normalized device coordinates:

We do this thing using a transformation that maintains the same relative placement of an object in normalized space as they had in viewing coordinates.

If a coordinate position is at the center of the viewing window:

It will display at the center of the viewport.

Fig shows the window to viewport mapping. A point at position (x_w, y_w) in window mapped into position (x_v, y_v) in the associated viewport.

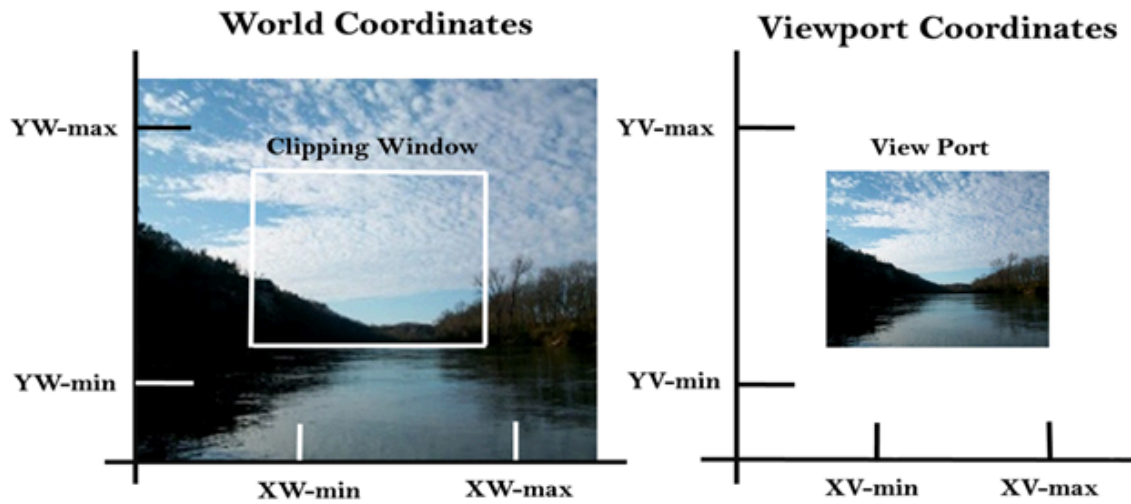
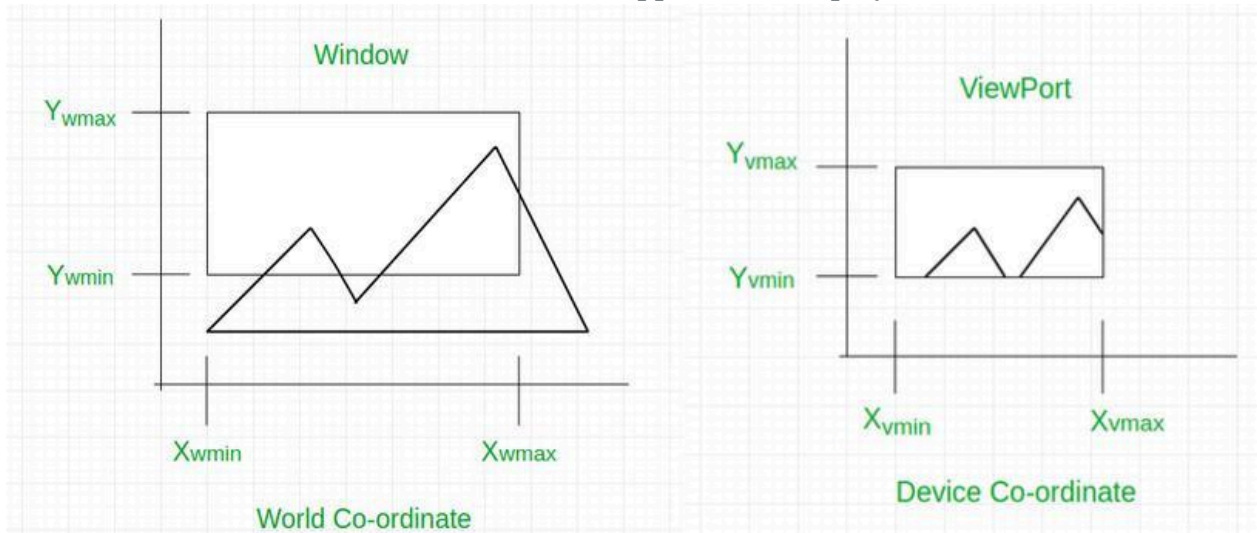


Fig: Window to viewport mapping

Window to Viewport Transformation is the process of transforming 2D world-coordinate objects to device coordinates. Objects inside the world or clipping window are mapped to the viewport which is the area on the screen where world coordinates are mapped to be displayed.



General Terms:

- **World coordinate** – It is the Cartesian coordinate w.r.t which we define the diagram, like X_{wmin} , X_{wmax} , Y_{wmin} , Y_{wmax}
- **Device Coordinate** –It is the screen coordinate where the objects are to be displayed, like X_{vmin} , X_{vmax} , Y_{vmin} , Y_{vmax}
- **Window** –It is the area on the world coordinate selected for display.
- **ViewPort** –It is the area on the device coordinate where graphics is to be displayed.

It may be possible that the size of the Viewport is much smaller or greater than the Window. In these cases, we have to increase or decrease the size of the Window according to the Viewport and for this, we need some mathematical calculations.

(x_w, y_w) : A point on Window

(x_v, y_v) : Corresponding point on Viewport

We have to calculate the point (x_v, y_v)

$$\text{Normalized Point on Window} \left(\frac{x_w - x_{wmin}}{x_{wmax} - x_{wmin}}, \frac{y_w - y_{wmin}}{y_{wmax} - y_{wmin}} \right)$$

$$\text{Normalized Point on Viewport} \left(\frac{x_v - x_{vmin}}{x_{vmax} - x_{vmin}}, \frac{y_v - y_{vmin}}{y_{vmax} - y_{vmin}} \right)$$

Now the relative position of the object in Window and Viewport are same.

For x coordinate,

$$\frac{x_w - x_{wmin}}{x_{wmax} - x_{wmin}} = \frac{x_v - x_{vmin}}{x_{vmax} - x_{vmin}}$$

For y coordinate,

$$\frac{y_w - y_{wmin}}{y_{wmax} - y_{wmin}} = \frac{y_v - y_{vmin}}{y_{vmax} - y_{vmin}}$$

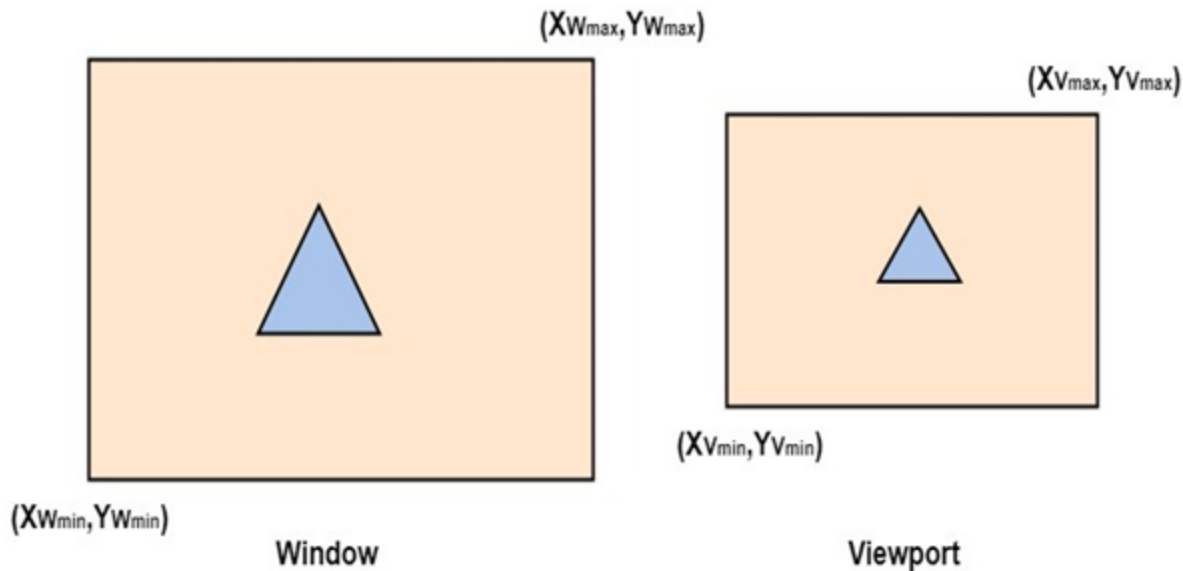
So, after calculating for x and y coordinate, we get

$$x_v = x_{vmin} + (x_w - x_{wmin}) S_x \quad y_v = y_{vmin} + (y_w - y_{wmin}) S_y$$

Where s_x is the scaling factor of x coordinate and s_y is the scaling factor of y coordinate

$$S_x = \frac{x_{vmax} - x_{vmin}}{x_{wmax} - x_{wmin}} \quad S_y = \frac{y_{vmax} - y_{vmin}}{y_{wmax} - y_{wmin}}$$

Matrix Representation of the above three steps of Transformation:



Step1: Translate window to origin 1

$$T_x = -X_{W_{min}} \quad T_y = -Y_{W_{min}}$$

Step2: Scaling of the window to match its size to the viewport

$$S_x = (X_{y_{max}} - X_{v_{min}}) / (X_{w_{max}} - X_{w_{min}})$$

$$S_y = (Y_{v_{max}} - Y_{v_{min}}) / (Y_{w_{max}} - Y_{w_{min}})$$

Step3: Again translate viewport to its correct position on screen.

$$T_x = X_{v_{min}}$$

$$T_y = Y_{v_{min}}$$

Above three steps can be represented in matrix form: $VT = T * S * T_1$

T = Translate window to the origin

S = Scaling of the window to viewport size

T_1 = Translating viewport on screen.

$$T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -X_{W_{min}} & -Y_{W_{min}} & 1 \end{bmatrix}$$

$$S = \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$T_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ X_{V_{min}} & Y_{V_{min}} & 1 \end{bmatrix}$$

Viewing Transformation = $T * S * T_1$

Advantage of Viewing Transformation:

We can display picture at device or display system according to our need and choice.

Note:

- o World coordinate system is selected suits according to the application program.
- o Screen coordinate system is chosen according to the need of design.
- o Viewing transformation is selected as a bridge between the world and screen coordinate.

Example: Let us assume,

- for window, $X_{wmin} = 20$, $X_{wmax} = 80$, $Y_{wmin} = 40$, $Y_{wmax} = 80$.
- for viewport, $X_{vmin} = 30$, $X_{vmax} = 60$, $Y_{vmin} = 40$, $Y_{vmax} = 60$.
- Now a point (X_w, Y_w) be $(30, 80)$ on the window. We have to calculate that point on the viewport i.e (X_v, Y_v) .
- First of all, calculate the scaling factor of x coordinate S_x and the scaling factor of y coordinate S_y using the above-mentioned formula.
$$S_x = (60 - 30) / (80 - 20) = 30 / 60$$
$$S_y = (60 - 40) / (80 - 40) = 20 / 40$$
- So, now calculate the point on the viewport (X_v, Y_v) .
$$X_v = 30 + (30 - 20) * (30 / 60) = 35$$
$$Y_v = 40 + (80 - 40) * (20 / 40) = 60$$
- So, the point on window $(X_w, Y_w) = (30, 80)$ will be $(X_v, Y_v) = (35, 60)$ on viewport.

REFERNCES

- D. Hearn & Baker: Computer Graphics with OpenGL, Pearson Education, Third Edition, 2009.
- Z Xiang, R A Plastock , P S Avadhani : Computer Graphics, Tata Mc GrawHill Education Pvt Ltd ., 2006
- <https://www.javatpoint.com/computer-graphics-window>
- <https://www.geeksforgeeks.org/window-to-viewport-transformation-in-computer-graphics-with-implementation/>
- <https://www.gatevidyalay.com>